Chapter 9 | DBMS & SQL

9.1 DBMS Basics, Data Models and Architecture

Database: an **organized collection of data that CAN be accessed, updated and managed** — it stores text, images, audio, video and files alike (all types). The **DBMS is the software that stores, modifies and retrieves the data**: it acts as the **interface between the DATABASE APPLICATION and the DATABASE**, and the classic textbook definition covers **all three** — interrelated data + access programs + enterprise description. **CRUD = Create, Read, Update, Delete** (→ INSERT, SELECT, UPDATE, DELETE). MySQL, Informix, SQL Server are **all databases; SQL Server is Microsoft's; SQL is the database language.**

(1) **Schema vs instance** (asked three times): the **schema is the overall DESIGN of the database**; the **instance is the STATE (data) at a given point of time. The structure of the data files is stored in the schema**; **metadata = data about data**, kept in the **data catalog / data dictionary** — the system database describing the data. A **data model** = the collection of concepts describing database structure.

(2) **Data models**: **hierarchical is the OLDEST** (a filing cabinet of customer folders is hierarchical), then network, then **relational** — defined by **TABLES** (rows/columns/structured format; **Dr E. F. Codd gave 12 rules** for a true RDBMS), then object-oriented; the **deductive model = INFERENCE model. 'Sequential' is NOT a DBMS category** (asked twice — it is a file organisation) and **'decentralized' is not a database type**. Design approaches: top-down, bottom-up, inside-out — **no 'up-down'. DDLC = Database Development Life Cycle**; **DASD = Direct Access Storage Device.**

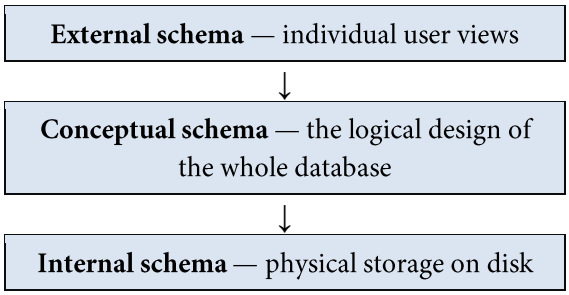


Fig. 9.1 — The three-level architecture. Logical data independence = changing the CONCEPTUAL schema without touching the EXTERNAL schemas; physical data independence = changing storage without touching the conceptual level.

(3) **Features & functions**: a DBMS supports ACID and high security but is built for many users — **'single-user access only' is NOT a feature**; its functions cover managing data and security — **'analysing code' is not one**; its utilities cover backup and data loading — **'process organization' is not one** (that is the OS). Database elements = schema, relationships, constraints — **not the FRONTEND. Data integrity = correctness and completeness of the data.** Applications: payroll, HR, invoicing — **digital drawing is not** one. **Denormalization = deliberately increasing redundancy to improve performance.**

(4) **Distributed databases**: **data allocation** decides which site stores what; **data fragmentation** breaks the database into logical units for the sites; the tested disadvantage is **data inconsistency. Applications talk to the server via ODBC or JDBC; web browsers are the universal front end**; the SQL naming hierarchy: **catalogs contain schemas** (which contain tables). JDBC's **REAL = single-precision** float (~7-digit mantissa).

(5) **Warehousing corner**: a data warehouse supports **OLAP** (analysis; OLTP stays on the operational DB); the **star schema = one central fact table + one table per dimension**; the **staging area** is the intermediate stop of ETL; **knowledge discovery and data mining are associated with the DATABASE.** Stray kept: computational-thinking order = **Abstraction → Decomposition → Pattern Recognition → Algorithm Design**.

Exam Points — 9.1

✦ DBMS = interface app↔database, stores/modifies/retrieves (all three textbook parts); database = organized + accessible collection; CRUD; SQL Server = Microsoft; SQL = database language.

✦ Schema = design, instance = state now (×3); metadata in the data catalog; hierarchical oldest; tables make it relational; Codd = 12 rules; deductive = inference; no sequential/decentralized/up-down; DDLC; DASD.

✦ Logical data independence = conceptual changes hidden from external views. Not features/functions: single-user only, analysing code, process organization, frontend. Integrity = correctness + completeness; denormalization adds redundancy for speed.

✦ Distributed: allocation, fragmentation, inconsistency disadvantage; ODBC/JDBC; browsers = universal front end; catalogs ⊃ schemas; REAL = single precision; warehouse = OLAP; star schema; staging area; mining ↔ database.

9.2 The Relational Model and Relational Algebra

(1) Codd's formal definition: a relation on domains $X_1 \ldots X_n$ is a **subset of the Cartesian product $X_1 \times X_2 \times \ldots \times X_n$.** **Tuple = row = record**; **attribute = column = field (a VERTICAL subset of the table)**; the **domain = the set of permitted values** (data type) of a column.

Degree and cardinality — asked five times with numbers:

$$Degree = number\ of\ ATTRIBUTES\ (columns) \qquad Cardinality = number\ of\ TUPLES\ (rows)$$

Worked: 500 tuples, 50 attributes → cardinality **500**. 50 tuples, 20 attributes → **50 / 20**. Degree 4, cardinality 3, then +3 attributes +4 tuples → **7 and 7**. Cartesian product STUDENT(20, 300) × TEACHER(10, 25): **degrees ADD (30), cardinalities MULTIPLY (7500)**; a natural join of |R| = 3 and |S| = 2 yields at most 3 × 2 = **6** tuples.

(2) **Algebra operations**: **R − S = the tuples in R but NOT in S** (difference); ∩ keeps common tuples — **intersection is BOTH commutative AND associative** (difference is neither); × pairs everything; **a JOIN needs two tables sharing a common column** (projection works on one table; Cartesian product needs no common column). The **result of a SELECT statement is itself a TABLE** — closure. A **crosstab query** summarises data from several tables into a grid. Storage aside: a **tablespace is broken into SEGMENTS** (then extents, then blocks).

Exam Points — 9.2

✦ Relation ⊆ $X_1 \times \ldots \times X_n$; tuple = row; attribute = vertical subset; domain = permitted values. Degree = columns, cardinality = rows: 500; 50/20; 7,7; product 30/7500; natural-join max 6.

✦ R–S = difference; intersection commutative + associative; join = two tables + common column; SELECT output = table; crosstab summarises; tablespace → segments.

9.3 The ER Model

(1) **Symbols** (the matching question: boxes–entities, ovals–attributes, diamonds–relationships): the values of the attributes **describe one particular ENTITY**; an **entity type** = the definition of entities sharing the same attributes

(the entity set is the current collection). A **relationship = an association among several entities**; its **degree = the number of participating ENTITY TYPES**; a **descriptive attribute records information about the RELATIONSHIP** itself.

ER notation	Meaning
Double rectangle / double box	**WEAK entity** (asked three times) — an entity set **without sufficient attributes to form a key**; its identifying relationship is a **double diamond**, the ratio strong : weak is **one-to-many**, and its partial key is the **DISCRIMINATOR** (full key = owner's PK + discriminator)
Ellipse family	Solid = stored attribute; **double ellipse = MULTI-VALUED**; **dashed ellipse = DERIVED** (asked twice — e.g. age from date of birth); underlined = key
Double lines	**TOTAL participation** (every entity takes part); single line = partial

(2) **Specialisation = the TOP-DOWN division of a higher-level entity into sub-entities** (asked twice); generalisation is the bottom-up reverse; aggregation treats a relationship as an entity. **One-to-many**: one teacher → many classes (each class one teacher). **Connection traps** are the ER-design pitfalls, and both **fan traps and chasm traps** are types. Mapping to tables: a **1:1 or 1:N relationship type is implemented by a FOREIGN KEY** (M:N needs a separate relationship table).

Exam Points — 9.3

- ✦ Box/oval/diamond = entity/attribute/relationship; attribute values describe an entity; relationship = association; degree = entity types; descriptive attributes → relationships.
- ✦ Weak entity: double rectangle (×3), no own key, 1:N from strong, discriminator partial key. Double ellipse = multivalued; dashed = derived (×2); double lines = total participation.
- ✦ Specialisation = top-down (×2), generalisation bottom-up; teacher→classes = 1:N; fan + chasm = connection traps; FK implements 1:1/1:N.

9.4 Keys and Integrity Constraints

The key ladder — asked in some form in ten papers:

$$Primary\ key \subseteq Candidate\ key \subseteq Super\ key$$

(1) A **superkey** = any attribute set that identifies a tuple uniquely — it need **NOT be minimal** (the false 'minimality' statement, twice). A **candidate key = a MINIMAL superkey**, and it **CAN be a primary key**; the **primary key is the candidate key chosen** (so 'primary key is also called candidate key'), the others become alternate/secondary keys — **a table can have only ONE primary key** (possibly composite) but **multiple secondary keys and multiple foreign keys**. The **PK = the unique identifier of each record** (asked five times); a student table's natural choice = **admission number**. **Nulls do NOT reduce space requirements** — the false RDBMS statement.

(2) **Foreign key = the column pointing to the PRIMARY KEY of ANOTHER table** — the key that LINKS tables (asked five times). **Referential integrity** (between two relations): every FK value must exist in the referenced relation; the two dangerous operations are **INSERT into the child (R2) and DELETE from the parent (R1)**. **Entity integrity:**

no primary key value can be NULL. Pairing: **foreign → referential, primary → entity**. Full access rights for all users = the integrity **breach**.

(3) **SQL constraints**: PRIMARY KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY — **UNION is NOT a constraint** (set operator). The **DEFAULT constraint provides a value for a column when none is specified**. A **primary index** = an ordered file of fixed-length records with two fields — the primary key and a block pointer.

Exam Points — 9.4

✦ PK ⊆ CK ⊆ SK; superkey never requires minimality; candidate = minimal superkey, promotable to primary; one PK per table, many secondary/foreign keys; PK = unique identifier (×5), never null; admission number; nulls don't save space.

✦ FK links tables and points at the other table's PK (×5); referential integrity between two relations — violated by child-INSERT and parent-DELETE; entity integrity = no null PK; foreign→referential, primary→entity; UNION not a constraint; DEFAULT supplies missing values; primary index = key + block pointer.

9.5 Normalisation and Functional Dependencies

Functional dependency: a relationship **between ATTRIBUTES**: in X → Y, the left side X is the **DETERMINANT** and the right side Y the **DEPENDENT** (two rows agreeing on X must agree on Y). **Armstrong's axioms = reflexivity, augmentation, transitivity; pseudo-transitivity is a DERIVED rule, not a primary axiom** — the answer keyed in both the DSSSB axiom questions.

Worked: Closure of PQ under F = {P → RT, Q → S, R → PS, QS → TU}: start {P, Q}; P → RT adds R, T; Q → S adds S; QS → TU adds U → closure = **PQRSTU** (every attribute).

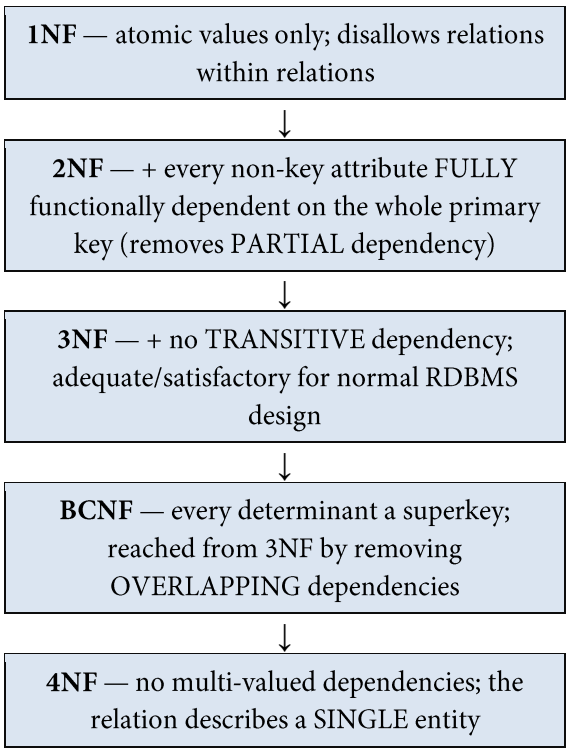


Fig. 9.2 — The normal-form ladder. Each form includes all the ones below it: a 2NF relation is definitely in 1NF.

(1) **2NF is the most-asked form** (six papers): defined by **full functional dependence on the (whole) primary key** — reached by **removing partial dependency**; partial dependency can only exist with a composite key. In the DSSSB

statement pair, only 'it is in 1NF' was correct — the second statement said 'independent' instead of 'dependent'. **3NF is the satisfactory/adequate form** for practical design (asked three times, incl. once in the DBMS-basics band). **BCNF: whenever a non-trivial X → A holds, X is a SUPERKEY.**

(2) **Higher normal forms mean LESS redundancy** — a 4NF table is MORE immune to inconsistencies than 3NF, and a 3NF table has FEWER anomalies than 2NF; the false statement in the KVS set is the claim that **'3NF is more vulnerable than 2NF'**. (The printed key marks option (a), but the book's own explanation identifies the reversed-redundancy claim — option (c) — as the false one; flagged for verification.) Recall from 9.1: **denormalization** deliberately walks back down this ladder for speed.

Note: Stray parked here from the source's normalisation band: the 2-D **viewing-pipeline order III → II → IV → I** (world scene → viewing → normalised → device coordinates) — a computer-graphics fact, covered in its own chapter.

Exam Points — 9.5

- ✦ FD relates attributes; X = determinant, Y = dependent; Armstrong = reflexivity/augmentation/transitivity (pseudo-transitivity derived, ×2); PQ^+ = PQRSTU.
- ✦ 1NF = atomic/no nested relations; 2NF = full dependence on the whole PK (×6; 2NF ⇒ 1NF); 3NF = no transitive, the adequate form (×3); BCNF = determinants are superkeys, from 3NF by removing overlapping dependencies; 4NF = single entity, no multivalued deps; higher NF = less redundancy (key/explanation mismatch flagged).

9.6 SQL Command Families — DDL, DML, DCL, TCL

Family	Commands and asked facts
DDL (structure)	**CREATE, ALTER, DROP, TRUNCATE** — defines relations, deletes relations, relates schemas (×2). **UPDATE is NOT DDL** (HTET); **INSERT is NOT DDL** (DSSSB); **DROP TABLE is the DDL pick** in the KVS set; **TRUNCATE is DDL** (NVS — not DML/DCL/TCL)
DML (data)	**SELECT, INSERT, UPDATE, DELETE** — provides query AND insert/delete/modify of tuples (×2, HTET both). **DELETE is the DML pick; INSERT INTO is DML** (KVS); SELECT alone is sometimes labelled DQL
DCL (rights)	**GRANT and REVOKE. GRANT is the DCL command** (Bihar STET); read permission: **GRANT SELECT ON Customers TO Deepak;**
TCL (transactions)	**COMMIT makes updates permanent** (×3 here + transactions band); ROLLBACK undoes; SAVEPOINT marks. Python-MySQL: **commit() needed after INSERT** (not after SELECT; DDL auto-commits)

(1) **Structure commands**: **ALTER TABLE adds, deletes or modifies COLUMNS** of an existing table (asked four times) — **ALTER TABLE TEACHER ADD COLUMN FEES FLOAT;** is the KVS-correct form; MODIFY TABLE / EDIT TABLE do not exist. CREATE TABLE X AS SELECT * FROM Y; builds and fills a copy in one statement. **DESC MANAGER;** shows the fields with their data types. **DROP of a schema with RESTRICT succeeds only if the schema is empty** (CASCADE drops the contents too).

Command	Effect on rows / structure

DELETE FROM t	Removes rows (all, or those matching WHERE); **table structure stays**; DML, can be rolled back. With an empty-result subquery in IN (…), **no rows are deleted** (not an error)
TRUNCATE TABLE t	**Removes ALL rows fast** (deallocates pages); **keeps the structure; is DDL and is FASTER than DELETE** — the one true statement in the 2025 set
DROP TABLE t	**Deletes the table permanently — data AND structure** (asked five times: DROP TABLE is the remove-table command)

(2) **Row commands**: **INSERT adds new rows** — the standard forms are INSERT INTO t VALUES (…) and **INSERT INTO t (cols) VALUES (…)** (the column-list form is the correct 2025 answer); **INSERT without INTO is the invalid SQL:1999 statement**. **UPDATE changes attribute values of selected tuples** (asked six times): **UPDATE … SET … WHERE** — workeds: UPDATE Customer SET Bill = Bill / 1.1 WHERE Account = 'High privilege'; and UPDATE STUDENT SET FEES = FEES * 1.1; (MODIFY/CHANGE/ASSIGN are not SQL). **SELECT retrieves data** (asked four times).

(3) **ROLLBACK to a savepoint** (the KVS NOT-true question): savepoints AFTER the target are erased, the target savepoint itself SURVIVES, resources are NOT freed — the false claim is that it **erases all modifications from the start of the transaction** (that is a full ROLLBACK).

Exam Points — 9.6

✦ DDL = CREATE/ALTER/DROP/TRUNCATE (UPDATE and INSERT are not); DML = SELECT/INSERT/UPDATE/DELETE (query + modify); DCL = GRANT/REVOKE; TCL: COMMIT permanent (×3), commit() after INSERT.

✦ ALTER TABLE adds/deletes/modifies columns (×4, TEACHER worked); CREATE TABLE AS SELECT; DESC shows structure; DROP … RESTRICT only when empty; GRANT SELECT ON Customers TO Deepak.

✦ DELETE = rows go, table stays (empty subquery → nothing deleted); TRUNCATE = all rows, fast, DDL; DROP = table + structure gone (×5); INSERT INTO column-list form (INSERT without INTO invalid); UPDATE SET WHERE (×6, both workeds); ROLLBACK-to-savepoint false claim = erases from start.

9.7 SELECT — Clauses, Patterns and Joins

The two orders — both asked:

$$\textit{Writing order}: \quad \textit{SELECT}, \textit{FROM}, \textit{WHERE}, \textit{GROUP BY}, \textit{HAVING}, \textit{ORDER BY}$$
$$\textit{Evaluation order}: \quad \textit{FROM}, \textit{WHERE}, \textit{GROUP BY}, \textit{HAVING}, \textit{SELECT}, \textit{ORDER BY}$$

(1) **WHERE filters ROWS (before grouping); HAVING filters GROUPS (after GROUP BY)** — asked four ways, including the KVS pair: 'HAVING needs GROUP BY' is TRUE, 'HAVING executes before WHERE' is FALSE. **GROUP BY gathers rows with the same value in a column**; worked: average salary of grade E4 → **GROUP BY grade HAVING grade = "E4"**. The **WITH clause creates a temporary relation** (CTE) for its query.

(2) **ORDER BY sorts** (SORTBY/ARRANGE do not exist); **ASC is the default** (asked three times). **Each column carries its OWN direction**: ORDER BY title, name DESC → titles ASCENDING, names within a title DESCENDING; writing the direction before the column (… ASC, DESC Name) is the INVALID statement. **DISTINCT prevents duplicate result rows** (asked twice; UNIQUE is a constraint, not a filter).

(3) **Pattern and range tests**: **LIKE matches character patterns** with **% = any run of characters, _ = exactly one** — 'contains INDIA/RPSC' → **LIKE '%INDIA%'** (asked four times). **BETWEEN 10 AND 20 is INCLUSIVE at BOTH ends. Nulls**: nothing equals NULL — use **IS NULL** (Address = NULL returns nothing); **NULL IS NULL → TRUE**, while NULL <= NULL → UNKNOWN. **SELECT** * retrieves all attributes; the **output of a SELECT is a table.**

(4) **Joins and subqueries**: a **JOIN combines data from multiple tables on a common column; a SUBQUERY is a query within a query; nested queries = complete select-from-where blocks inside another query**; a **correlated subquery** references the outer row — the dept-average worked answer aliases the outer table (s) so the inner AVG recomputes per department. A **SELF JOIN joins a table to ITSELF** (aliases required). **FULL JOIN returns all rows from both tables with NULLs for non-matches** (asked twice).

Worked: Second-highest result: **SELECT MAX(result) FROM student WHERE result < (SELECT MAX(result) FROM student);** — the portable nested-MAX pattern. **Salary >= ALL (SELECT salary …)** → the maximum earners. The KVS join query: SELECT SUBNAME, COUNT(*) FROM SUBJECTS NATURAL JOIN RESULT GROUP BY SUBNAME;. The UNION query returns **all projects involving employee 'XXXX' as worker OR as manager.**

(5) **Views**: a view = a **VIRTUAL table derived from other tables** (asked three times; the base table stores real rows). A SQL:1999 view definition may use subqueries and aggregates but **NOT ORDER BY**; **WITH CHECK OPTION ensures inserted/updated rows satisfy the view's WHERE clause**. The three-way 'wrong statements' set: SQL does **not** auto-eliminate duplicates, works **without** indexes, and forbids repeated attribute names — **all three claims wrong.**

Exam Points — 9.7

✦ Writing vs evaluation order (both memorised); WHERE = rows before grouping, HAVING = groups after (×4); GROUP BY groups equal values; E4 worked; WITH = temporary relation.

✦ ORDER BY, ASC default (×3); direction follows its own column (worked pair); DISTINCT (×2); LIKE %/_ → '%INDIA%' (×4); BETWEEN inclusive; IS NULL tests (= NULL never true; NULL IS NULL = TRUE).

✦ Join = combine tables / subquery = nested query; correlated subquery aliases the outer row; self join = itself; FULL JOIN keeps all + NULLs (×2); nested-MAX second-highest; >= ALL = maximum; NATURAL JOIN + GROUP BY worked; UNION worked; views virtual (×3), no ORDER BY, WITH CHECK OPTION.

9.8 Functions, Operators and Data Types

(1) **Aggregates: COUNT, SUM, AVG, MIN, MAX** — the valid set is COUNT, MAX, AVG, SUM; **CROSS PRODUCT is not an aggregate**, and among Minus/Intersect/Union/Count only **COUNT is the aggregate** (the rest are set operators). COUNT(*) counts all tuples (SELECT COUNT(*) FROM Book); **COUNT(column) and AVG skip NULLs** — the KVS COACH worked: COUNT(GAME) = **3**, AVG(SALARY) = (60000 + 80000)/2 = **70000. MIN(Price) fetches the lowest value.**

(2) **Scalar functions**: **CEILING = the smallest integer ≥ the value** (FLOOR rounds down); **ROUND(65.726, –1) = 70** (negative precision rounds to tens); **GETDATE() returns the current datetime without time-zone offset**; **DATEDIFF: statement I only** — it reports the start/end difference, but datepart must be a KEYWORD, not a variable.

(3) **Operators**: concatenation = || (double pipe); **>= is the comparison operator** (+ − / are arithmetic); **=> is INVALID** (asked twice — <>, <=, = are fine). Comments: -- and /* … */ only; // is NOT valid SQL.

(4) **Data types**: DATE and BOOLEAN are standard SQL:1999; **STRING and ENUM are NOT** (ENUM is MySQL-only — asked twice); REAL, INTERVAL, TIMESTAMP are valid but **BASE is not a SQL type**. An **OLE object field embeds objects made in other Windows applications**.

Exam Points — 9.8

✦ Aggregates = COUNT/SUM/AVG/MIN/MAX (not CROSS PRODUCT; not the set operators); COUNT(*) = tuples; NULL-skipping worked 3 / 70000; MIN(Price).

✦ CEILING rounds up; ROUND(65.726, −1) = 70; GETDATE(); DATEDIFF only I; || concatenates; >= valid, => invalid (×2); comments -- and /* */ only; ENUM/STRING/BASE not SQL types; OLE object field.

9.9 Transactions, ACID and Concurrency Control

ACID = Atomicity, Consistency, Isolation, Durability — asked in four papers, and **'Instance' is the non-member** in the DSSSB set. **COMMIT makes a transaction's changes permanent** (durability begins at commit). A **TIMESTAMP** uniquely identifies a transaction's relative starting time; timestamp-ordering variants = **total, partial, multiversion — 'multilevel' is NOT one**.

(1) **Locks maintain transactional integrity and consistency** (the Bihar STET answer). **Shared lock = read only** (statement I valid); **exclusive lock = read AND write** (so 'T2 cannot write R' was false). **Two-phase locking: lock requests in the GROWING phase, unlocks in the SHRINKING phase. Concurrency control matters for UPDATES in a MULTI-USER environment**; storing one fact twice → **wasted space AND inconsistency** (both).

(2) **Anomalies**: **lost update = both transactions read X, then both write** (asked twice — the R1 R2 W1 W2 trace); **unrepeatable read = the read-write conflict**; dirty read = reading uncommitted data. **Conflict serializability is tested with the precedence graph**: in the 2023 worked pair S1's graph is cyclic (NOT serializable) and **S2 is conflict serializable**. A **STRICT schedule**: no one reads or writes X until the last writer of X commits or aborts.

(3) **Recovery**: the **CHECKPOINT is the synchronisation point between the database and the log** (asked twice — it limits the recovery work after a crash); **shadow paging is the alternative to log-based recovery**. Deadlock prevention: **wait-die is NON-preemptive, wound-wait is PREEMPTIVE. OLTP: immediate responses + always up-to-date database (I and III); batch processing: only scheduled-interval reports (III)** — it is low-cost and updates only at run time. Crash-recovery duplication is prevented by **sequence numbers and acknowledgments**.

Note: Four commerce strays from this band, kept as reference: **NEFT has all four listed advantages**; the recording/classifying/summarising-in-money definition = **financial accounting**; revenue income minus expense = **profit**; double-entry = **debit and credit** entries.

Exam Points — 9.9

✦ ACID (×4; Instance not a member); COMMIT = permanence; timestamp = relative start time; variants total/partial/multiversion (no multilevel); locks maintain integrity; shared = read, exclusive = read + write; 2PL growing/shrinking.

✦ Lost update = read-read-write-write (×2); unrepeatable read = read-write conflict; precedence graph → S2 serializable; strict schedule waits for the last writer; checkpoint = DB↔log sync (×2); shadow paging = non-log recovery; wait-die non-

preemptive / wound-wait preemptive; concurrency = multi-user updates; duplication = waste + inconsistency; OLTP I + III; batch III only; sequence numbers stop crash duplicates.

9.10 Indexing and File Organisation

(1) **B-tree index: RANGE queries, hierarchical sorted order; hash index: EXACT-match lookups via a hash function** — the 2023 comparison answer. **ISAM = Indexed Sequential Access Method**. A **primary index built on the ordering key is the CLUSTERING index** (sparse — one entry per block, as in 9.4).

(2) **Sequential file organisation: BOTH statements true** — insertion/deletion forces the whole file to be rewritten, and finding a record scans **half the file on average**. **File volatility is HIGH when data is frequently added and deleted**. Common file operations = create, delete, rename — **all of them** (an OS fact kept here as asked).

(3) **Fragmentation family**: **data fragmentation** = breaking a database into logical units for distributed sites (9.1); **DEFRAGMENTATION** = relocating small non-contiguous disk blocks into contiguous ones. The **staging area** is the ETL intermediate store; the customer filing cabinet = **hierarchical**.

Note: One network stray from the SQL band, parked: the firewall matching **a-iv, b-ii, c-iii, d-i** (stateful–inbound traffic, packet filter–OS, proxy–mail server, application-level–Telnet) — Chapter 10 territory. Also from the DDL band: the operator matching **1-iii, 2-i, 3-ii**.

Exam Points — 9.10

✦ B-tree = range/hierarchical, hash = exact match; ISAM; primary-on-ordering-key = clustering index; sequential org: rewrite on insert/delete + half-file average scan (both true); volatility high with frequent add/delete.

✦ Data fragmentation (distributed) vs disk defragmentation; staging area = ETL stop; filing cabinet = hierarchical; file ops = create/delete/rename (all); firewall and DDL-operator matchings parked.

Quick Revision — 35 points

1. DBMS = software that stores, modifies and retrieves data; the interface between the database application and the database; textbook definition = data + programs + enterprise. Database = organized, accessible, updatable collection. CRUD = Create, Read, Update, Delete.
2. Schema = the design; instance = the state at a point in time. Metadata = data about data, stored in the data catalog / dictionary. Data model = concepts describing structure.
3. Models: hierarchical oldest (filing cabinet), network, relational (tables; Codd's 12 rules), object-oriented, deductive = inference. Not real categories: sequential, decentralized, up-down approach. DDLC; DASD = Direct Access Storage Device.
4. Three levels: external, conceptual, internal. Logical data independence = conceptual changes without touching external views; physical = storage changes hidden. Catalogs contain schemas.
5. Not DBMS features/functions: single-user-only access, analysing code, process organization, frontend as an element. Integrity = correctness + completeness. Denormalization = add redundancy for speed. Distributed: allocation, fragmentation, inconsistency disadvantage. ODBC/JDBC; browsers = universal front end.
6. Warehouse = OLAP; star schema = one fact table + dimension tables; staging area = ETL stop; data mining/KDD ↔ database. JDBC REAL = single precision.
7. Relation $\subseteq X_1 \times \ldots \times X_n$. Tuple = row; attribute = column (vertical subset); domain = permitted values. Degree = columns, cardinality = rows: workeds 500; 50/20; 7,7; product degrees add / cardinalities multiply (30, 7500); natural-join max $|R| \times |S|$ = 6.

8. R–S = in R not S; intersection commutative and associative; join needs a common column; SELECT output is a table; crosstab summarises; tablespace → segments → extents → blocks.
9. ER symbols: box entity, oval attribute, diamond relationship. Weak entity = double rectangle, no own key, 1:N from its strong owner, discriminator = partial key. Double ellipse multivalued; dashed derived; double lines total participation.
10. Relationship = association among entities; its degree = participating entity types; descriptive attributes describe the relationship. Specialisation top-down, generalisation bottom-up; fan and chasm = connection traps; FK implements 1:1/1:N.
11. PK ⊆ CK ⊆ SK. Superkey unique but not minimal; candidate = minimal superkey; primary = the chosen candidate (one per table, never null); others = alternate/secondary. Admission number = student PK. Nulls do not save space.
12. FK = column pointing at another table's PK; links tables. Referential integrity: child-INSERT and parent-DELETE are the risky operations. Entity integrity: PK never NULL. Foreign→referential, primary→entity.
13. Constraints: PRIMARY KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY (UNION is not). DEFAULT supplies missing values. Primary index = key + block pointer, sparse; on the ordering key it is the clustering index.
14. FD is between attributes; X → Y: X determinant, Y dependent. Armstrong = reflexivity, augmentation, transitivity; pseudo-transitivity is derived. PQ^+ under the given F = PQRSTU.
15. Ladder: 1NF atomic (no nested relations) → 2NF full dependence on the whole PK (no partial) → 3NF no transitive (adequate for practice) → BCNF determinants are superkeys (remove overlapping dependencies) → 4NF single entity, no multivalued deps.
16. 2NF ⇒ definitely 1NF. Higher normal form = less redundancy (the '3NF more vulnerable than 2NF' claim is the false one; source key/explanation mismatch flagged). Viewing-pipeline stray: III→II→IV→I.
17. DDL = CREATE/ALTER/DROP/TRUNCATE; DML = SELECT/INSERT/UPDATE/DELETE; DCL = GRANT/REVOKE; TCL = COMMIT/ROLLBACK/SAVEPOINT. UPDATE and INSERT are never DDL; TRUNCATE is DDL.
18. COMMIT makes changes permanent; Python-MySQL needs commit() after INSERT. ROLLBACK-to-savepoint: later savepoints erased, the target survives, resources stay held — it does NOT erase everything from the start.
19. ALTER TABLE adds/deletes/modifies columns: ALTER TABLE TEACHER ADD COLUMN FEES FLOAT;. CREATE TABLE X AS SELECT * FROM Y;. DESC t shows fields + types. DROP … RESTRICT only if empty; CASCADE drops contents.
20. DELETE FROM = rows go, table stays (empty IN-subquery deletes nothing); TRUNCATE = all rows fast, structure stays, faster than DELETE; DROP TABLE = data + structure gone.
21. INSERT INTO t (cols) VALUES (…); INSERT without INTO is invalid. UPDATE t SET col = expr WHERE cond — Bill/1.1 High-privilege and FEES*1.1 workeds; MODIFY/CHANGE/ASSIGN do not exist. GRANT SELECT ON Customers TO Deepak;.
22. Writing order SELECT FROM WHERE GROUP BY HAVING ORDER BY; evaluation order FROM WHERE GROUP BY HAVING SELECT ORDER BY.
23. WHERE filters rows before grouping; HAVING filters groups after; HAVING needs GROUP BY (true), HAVING-before-WHERE (false). E4 worked: GROUP BY grade HAVING grade = "E4". WITH = temporary relation (CTE).
24. ORDER BY sorts, ASC default; each column its own direction (title ASC, name DESC); direction never precedes the column. DISTINCT removes duplicate rows.
25. LIKE with % (any run) and _ (one char): contains → '%INDIA%'. BETWEEN inclusive both ends. = NULL never true; use IS NULL; NULL IS NULL = TRUE, NULL <= NULL = UNKNOWN. SELECT * = all attributes.
26. Join combines tables on a common column; subquery = query within a query; nested queries = full select-from-where blocks inside another; correlated subquery aliases the outer row (dept-average worked); self join = a table with itself; FULL JOIN = everything + NULLs.
27. Second-highest = nested MAX with < overall MAX. Salary >= ALL (…) = the top earners. NATURAL JOIN + GROUP BY for per-subject counts. The UNION query = projects with 'XXXX' as worker or manager.
28. Views = virtual tables derived from others; no ORDER BY inside a SQL:1999 view; WITH CHECK OPTION forces writes to satisfy the view's WHERE. SQL does not deduplicate automatically, runs without indexes, forbids repeated attribute names.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

29. Aggregates COUNT/SUM/AVG/MIN/MAX; CROSS PRODUCT and the set operators are not. COUNT(*) counts tuples; COUNT(col)/AVG skip NULLs — COACH worked 3 and 70000.

30. CEILING = smallest integer ≥; ROUND(65.726, −1) = 70; GETDATE() = current datetime; DATEDIFF statement I only. || concatenates; >= valid, => invalid; comments -- and /* */ only.

31. SQL:1999 types: DATE, BOOLEAN, REAL, INTERVAL, TIMESTAMP valid; STRING, ENUM (MySQL-only), BASE invalid. OLE object fields embed Windows-application objects.

32. ACID = Atomicity, Consistency, Isolation, Durability (Instance is not one). Timestamp = relative start marker; ordering variants total/partial/multiversion (no multilevel). Locks preserve integrity; shared = read, exclusive = read + write; 2PL = growing locks, shrinking unlocks.

33. Lost update = both read then both write; unrepeatable read = read-write conflict; dirty read = uncommitted data. Precedence graph: acyclic ⇔ conflict serializable (S2 yes, S1 no). Strict schedule waits for the last writer's commit/abort.

34. Checkpoint = DB↔log synchronisation, bounds recovery; shadow paging = the non-log alternative. Wait-die non-preemptive, wound-wait preemptive. OLTP = immediate + up-to-date (I, III); batch = scheduled reports only (III); sequence numbers + acks stop crash duplicates.

35. B-tree index = ranges, hierarchical; hash = exact match. ISAM = Indexed Sequential Access Method. Sequential org: full rewrite on insert/delete, half-file average search; volatility high with frequent change. Data fragmentation (distributed) vs defragmentation (disk); filing cabinet = hierarchical.